scan for the full sheet

← 04 · Two boots | **Step 6 of 17** · Make the three keys | 07 · Build the 2-of-3 →

Make the three keys

Time	Difficulty	Steps	You will need
30-45 min	Moderate	4 sections	5 dice · Paper and pen · The worksheet

Your path

Easy

One key

Full program

Your path: **Full program**

Not on your path

Switch to Full program to read this sheet

Risks of your path →

```
11
01 00 10 01 01
11 11 00 10 00 10
00 01
10
```

256-BIT STRING

```
11 00 10 01 01 01 11 00 10 11 00 10 00 01 10
```

…keep rolling… 256 / 256 BITS

> **WARNING**
>
> Never enter seed words on an online computer.

This work starts after Tails has booted with networking set to Disabled, and before any plate is stamped. Five casino dice are required, and every face has to be readable. Faces 1 through 4 are kept. Faces 5 and 6 are rolled again. The work is done three times, for three keys, and the computer is shut down fully between them. A private room, no camera.

The faces

- 1 is 00. 2 is 01. 3 is 10. 4 is 11.
- 5 and 6 are not bits. Roll those dice again.
- Read the dice left to right, the same way every shake.
- Example: 4, 6, 1, 3, 2 becomes 11, skip the 6, then 00, 10, 01.
- Write only 0 and 1. Roll exactly 256 bits (128 pairs, about 39 shakes), then stop. More bits do not add security (everything folds down to 256), and extra bits make the two tools below disagree, which stops the work for no reason.

Stop if a check fails. Do not skip ahead.

Plebum Dominium · plebumbtc.io · Version 2026.10.27 · Built October 27, 2026
Content SHA-256: f5886c372356b029aac98aadaa44cc388a89c2bb9f7873fd7e6b47e472e83222

scan for the full sheet

Turn the bits into 24 words

1. 01 · Boot Tails. Networking Disabled. Copy the checked converter and the Ian Coleman page from the data USB to the Desktop.
2. 02 · Run the converter. Paste the bits. It prints 24 words. Word 24 is the check. Do not invent it. Do not “fix” it.
3. 03 · Open the Ian Coleman page in the offline browser. Entropy type: Binary. 24 words. Paste the same bits. The 24 words must match. If they differ, stop. Do not pick the prettier list.
4. 04 · Write the 24 words on the grid. Read them back out loud once.
5. 05 · Shut Tails down. Wait until the screen is dark. Then start the next key. Seed 2 is a new set of rolls. Seed 3 is a new set of rolls. Never continue a list from the previous boot.

```
$ cd ~/Desktop$ python3 bip39_dice_converter.py
```

The last word is a check

The 24 words are not 24 free choices. The last word carries a short check so a copying mistake is caught before metal is stamped. At least two of these checks are used. If they disagree, the work stops.

- Two tools, same bits, same 24 words. Converter, then the offline Ian Coleman page. Do not pick the prettier list.
- Let the wallet refuse a bad list. Sparrow or Electrum, networking off, will open a valid list and reject a broken one. If it will not open, do not change a word to force it.
- The math, if it needs to be seen. Twenty-four words are 264 bits. The first 256 are the dice. The last 8 are the check. Only word 24 can change to make that true. It is not chosen. The converter appends it after the file has been checked.

Practice grid

Use the printed recording grid in the bench sheet PDF. Each seed needs about 39 shakes for 256 bits. Count the pairs as you go and stop at 128.

Next, Build the 2-of-3.

What happens next

Once the words are made, the path forks. Pick what happens next.

Easy

One key

Full program

← Less work, more trust

More work, less trust →

Why about 39 shakes: the How many dice rolls page lists the old counts and the corrected math.

(function () { var PATHS = { easy: { cls: "path-card-easy", sel: "sel-easy", title: "Into one trusted device", trade: "Trust: one device and its maker. Work: lowest.", steps: ["Stamp the words in titanium first.", "Enter the words on the hardware wallet itself.", "Done. The device holds the wallet."], risk: "Risk: one device, one maker. If either fails you, the wallet fails.", go: "/handouts/single-sig.html", goText: "Go to the single-sig sheet" }, one: { cls: "path-card-one",

Content SHA-256: f5886c372356b029aac98aadaa44cc388a89c2bb9f7873fd7e6b47e472e83222

scan for the full sheet

sel: "sel-one", title: "One key plus a passphrase", trade: "Trust: one key, one device, one passphrase. Work: medium.", steps: ["Add a passphrase, kept on paper apart from the plate.", "Stamp the words in titanium.", "Open a watch-only wallet. The device signs."], risk: "Risk: one key and one passphrase. Lose the passphrase and the wallet is gone.", go: "/handouts/single-sig.html", goText: "Go to the single-sig sheet" }, full: { cls: "path-card-full", sel: "sel-full", title: "Three keys, three places", trade: "Trust: no single key, device, or place. Work: highest.", steps: ["Shut down fully. Repeat for two more keys.", "Build the 2-of-3. Practice spend first.", "Stamp three plates. Hide the encrypted map."], risk: "Risk: the most steps to get right. Lose two keys and the map and the funds are locked.", go: "/handouts/sparrow.html", goText: "Go to Build the 2-of-3" } }; var card = document.getElementById("path-card-dice"); var buttons = document.querySelectorAll("#path-toggle .path-seg button"); function show(path) { var p = PATHS[path]; card.className = "path-card " + p.cls; var html = "<h3>" + p.title + "</h3><p class=\"path-trade\">" + p.trade + " "; p.steps.forEach(function (s) { html += "
" + s + "
"; }); html += " " + p.risk + " " + p.goText + " → "; card.innerHTML = html; buttons.forEach(function (b) { var on = b.getAttribute("data-path") === path; b.setAttribute("aria-pressed", on ? "true" : "false"); b.className = on ? PATHS[b.getAttribute("data-path")].sel : ""; }); } buttons.forEach(function (b) { b.addEventListener("click", function () { show(b.getAttribute("data-path")); }); }); show("full"); })();

Content SHA-256: f5886c372356b029aac98aadaa44cc388a89c2bb9f7873fd7e6b47e472e83222